Deploying Qwen3-1.7B on iPhone with Apple Core AI: A Static W8 Implementation and Same-Device Profile Evaluation

Yi Shi
Independent Researcher
massif0601@gmail.com
ORCID: 0009-0002-7894-2703

Mengni Wu
Independent Researcher
mengninolab@gmail.com
ORCID: 0009-0004-7465-9067

September 1, 2026

Abstract

Apple's public `coreai-models` repository contained no Qwen3-1.7B preset at the revision examined in this study. We implement a version-pinned static iOS path with per-tensor W8 K-means-palettized transformer projections, a separately INT8-quantized tied embedding, FP16 compute, fixed KV state, a 4,096-token context, and `h16p` ahead-of-time compilation with Neural Engine-preferred compute. A formal authoring-model evaluation under the amended fidelity-v2 protocol achieved 0.996897 mean and 0.953391 minimum logit cosine on the holdout, 96.97% top-1 agreement, and a 0.008880 mean candidate-minus-reference NLL delta. On an iPhone 15 Pro running iOS 27, the historical W8 artifact completed four six-case suites. An Instruments trace recorded 547 MPSGraph program intervals and 547 Apple Neural Engine Prediction intervals, supporting ANE participation in that traced run. A separately published W8 rebuild passed a six-case suite in its recorded environment. A later load-only diagnostic compared that earlier public AOT with a current-toolchain build in one iOS 27 environment: the former failed twice during offline compilation, while the latter loaded and unloaded once. Because both the exported bytes and compiler producer changed, this diagnostic does not isolate a cause. The paired quality comparison uses the historical Static-W8 artifact and a Dynamic-INT4 profile on a frozen 300-example CMRC2018 subset. Static-W8 scored 59.33 EM / 81.70 F1 and Dynamic-INT4 scored 57.67 EM / 81.42 F1; both paired confidence intervals included zero. We then compared the exact current public W8 and INT4 artifacts in an eight-block speed run containing 120 completed profile/workload observations and 60 paired logical samples. Dynamic-INT4 used less storage and peak process memory and decoded faster, while Static-W8 reached the first token sooner and completed the near-4K workload in less time. Because the profiles differ in several coupled design choices, these measurements compare complete system profiles rather than isolated precision or compute-unit effects.

1 Introduction

Deploying a language model on a phone requires more than compatible weights. Weight representation, KV-cache allocation, graph shape, compilation target, runtime sessions, and memory release jointly determine whether the model can be compiled and used on a physical device.

Qwen3 provides dense and mixture-of-experts language models with unified thinking and non-thinking behavior (Yang et al., 2025). This report concerns the dense Qwen3-1.7B checkpoint at a fixed upstream revision (Qwen Team, 2025). Both profiles studied here derive from revision `70d244cc86ccca08cf5af4e1e306ecf908b1ad5e` and publish the same tokenizer SHA-256.

At Apple `coreai-models` commit `7062017c8e86c6cf4f49b721ddc3494efcdb7c7d`, the Qwen3 support table and model registry listed 0.6B, 4B, and 8B presets but no 1.7B preset. Apple issue 116 remained open as of August 28, 2026 (Apple Inc., 2026c). Pull request 196 proposed macOS and iOS support but closed without merge after its maintainer reported a local iOS regression; the comment did not identify its cause (Apple Inc., 2026d).

Community Core AI artifacts already cover dynamic GPU and stateful INT8 routes, with different devices, graph forms, validation methods, and published materials. We implement a version-pinned static iOS authoring path for Qwen3-1.7B, compile it for `h16p`, run it through `LanguageModelSession`, and compare it with a dynamic profile on the same device.

This report makes four contributions:

1. A version-pinned Apple-main patch, compression recipe, metadata, focused tests, export instructions, and `h16p` AOT procedure for an experimental Qwen3-1.7B iOS preset.
2. Frozen conversion-fidelity results, repeated physical-device suites for distinct W8 artifacts, an Instruments trace showing ANE participation, and a later load-only compatibility diagnostic.
3. A same-device evaluation against Dynamic-INT4 with per-example predictions, paired uncertainty estimates, per-run timing records, storage measurements, and process RSS.
4. Public recipes, a source patch, model artifacts, iOS sample applications, recorded hashes, and the deterministic pipeline used to reproduce the reported numbers.

2 Background and Related Work

2.1 Core AI authoring and runtime

Apple's public `coreai-models` repository describes export recipes, reusable authoring primitives, and Swift runtime utilities for integrating `.aimodel` resources with Core AI on macOS and iOS (Apple Inc., 2026a). The repository revision examined here requires version-27 operating systems and Xcode and publishes a registry of model presets. Its contribution policy at that revision states that code pull requests were not being accepted at launch. We therefore present the patch as a reproducible reference implementation, not as an upstream contribution accepted by Apple.

Apple has separately documented transformer deployment on the Apple Neural Engine and model palettization (Apple Machine Learning Research, 2022; Apple Inc., 2026b). Those publications provide the general technical context for static-shape and compressed execution. ANE participation in our implementation is supported by the recorded Core AI trace.

2.2 Qwen3-1.7B public status

Table 1 distinguishes Apple's dated registry state, two community artifacts, and the two profiles evaluated in this work. The repository named `qwen3-1.7b-CoreAI-official` is a community Hugging Face repository, not an Apple publication. Its cited model-card revision reports a dynamic INT4 GPU bundle compiled for `h18p` and measurements on iPhone 17 Pro; we do not reuse those measurements in the same-device comparison (mlboydaisuke, 2026). A second community repository publishes a stateful INT8 `.aimodel`; its reproduction manifest uses the macOS export path, and its numeric accuracy gate is recorded as not run (kevinqz, 2026).

Work	Route	Public evidence	Boundary
Apple public Qwen3 registry	Official authoring presets	Qwen3-0.6B, 4B, and 8B; no 1.7B preset at the audited commit	Repository state only at commit 7062017c
Apple issue 116 / PR 196	Open request / closed attempt	Issue open; PR closed and unmerged after a reported local iOS regression	No regression root cause, acceptance, or roadmap commitment
mlboydaisuke community artifact	Dynamic INT4 / GPU / h18p	iPhone 17 Pro timing and eight model-card questions	Different device, context, and evaluation protocol
kevinqz community artifact	Stateful INT8 .aimodel; macOS export manifest	Structural gate passed; numeric gate not run	No disclosed iPhone AOT or paired benchmark
This work: W8 profile	Static W8 / fixed KV / ANE-preferred / h16p	Frozen fidelity gate, iPhone 15 Pro suites, and ANE trace participation	One device class; no exclusive-ANE claim
This work: INT4 profile	Dynamic INT4 / growing KV / GPU-preferred / h16p	Same-device paired quality, latency, storage, and RSS comparison	Coupled system profile; no isolated causal attribution

Table 1: Public Qwen3-1.7B Core AI status audited 2026-08-28. Community model-card claims are contextual, not re-measured results.

Our evaluation focuses on a static `h16p` route on A17 Pro and a controlled comparison with a dynamic route on the same device.

3 System Profiles

3.1 Shared source identity

The W8 and INT4 artifacts originate from `Qwen/Qwen3-1.7B` revision `70d244cc86ccca08cf5af4e1e306ecf908b1ad5e`; their manifests record tokenizer SHA-256 `aeb13307a71acd8fe81861d94ad54ab689df773318809eed3cbe794b4492dae4`. Both cap the evaluated context at 4,096 tokens and target the `h16p` architecture, but most remaining deployment dimensions differ.

Dimension	Static-W8	Dynamic-INT4
Transformer weights	W8 per-tensor K-means palettization	INT4 block-32 symmetric-with-clipping
Embedding	INT8 per-tensor quantization	not separately reported in the artifact summary
Compute	FP16	FP16
KV cache	FP16 fixed-size	FP16 growing/dynamic
Graph	static	dynamic
Context cap	4,096 tokens	4,096 tokens
Preferred compute	neural-engine	gpu
AOT target	h16p	h16p

Table 2: Compared deployment profiles. The rows are coupled system choices, not independently randomized factors.

3.2 Static-W8 profile

Static-W8 begins from the unquantized upstream checkpoint. The recipe applies per-tensor, eight-bit K-means palettization to 196 transformer projection weights while excluding the embedding modules from that pass. The iOS authoring path quantizes the tied embedding separately to per-tensor INT8.

The reference patch adds a Qwen3-1.7B model preset and metadata, introduces the version-pinned recipe, marks the preset experimental, adds focused registry and export-contract tests, and extends iOS conversion-matrix coverage. Against Apple main commit `04a3fd6cfe9bfae9cf05b1f246cf915d930d1c0a`, the patch applied cleanly; 28 focused tests passed, 42 related iOS conversion tests collected without errors, Ruff passed, and the dry-run resolved FP16 compute, the W8 recipe, 4,096 tokens, and enabled embedding quantization.

For a subsequent compatibility diagnostic, we repeated the full export and `h16p` AOT compilation at the same Apple commit under Xcode 27 Beta 6. This third artifact identity was first observed in the load-only diagnostic, then published as `A-W8-CURRENT` and used in the separate speed-v2 generation run. Appendix A records its authoring and compiled hashes together with a second export from the same pinned inputs.

The exported `.aimodel` is compiled with `coreai-build` for iOS 27, `h16p`, and Neural Engine-preferred compute. The historical compiled artifact exposes 34 AOT functions across prompt, context, extension, and query buckets. The runtime loads the resource through `CoreAILanguageModel` and performs generation using `LanguageModelSession`.

Artifact identities. We distinguish three W8 binaries throughout: `A-W8-HISTORICAL`, used in the historical device suites and paired quality comparison; `A-W8-JULY-PUBLIC`, the July downloadable rebuild with its own six-case suite; and `A-W8-CURRENT`, first used as the current-toolchain side of the load diagnostic and subsequently published and measured in the speed-v2 run. Evidence is never transferred from one identity to another.

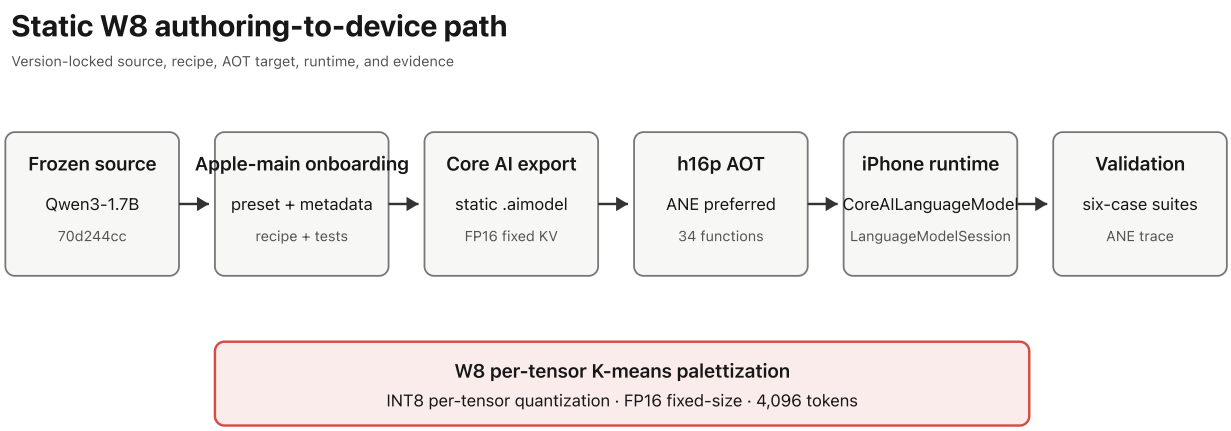

Figure 1: Version-pinned Static-W8 authoring-to-device path.

3.3 Dynamic-INT4 profile

Dynamic-INT4 uses block-32 symmetric-with-clipping INT4 transformer weights, FP16 compute, a growing FP16 KV cache, a dynamic graph, and GPU-preferred `h16p` compilation. On the test device, the runtime selected the `coreai-pipelined` engine and reported GPU compute.

An early pilot relied only on the `/no_think` prompt suffix. Inspection showed that this did not produce the same template state for both profiles, so the pilot was invalidated and its outputs were excluded. Before the accepted 300-pair run, the runtime was amended to pass `enable_thinking=false` to the model's chat template; that control was then held constant across both profiles.

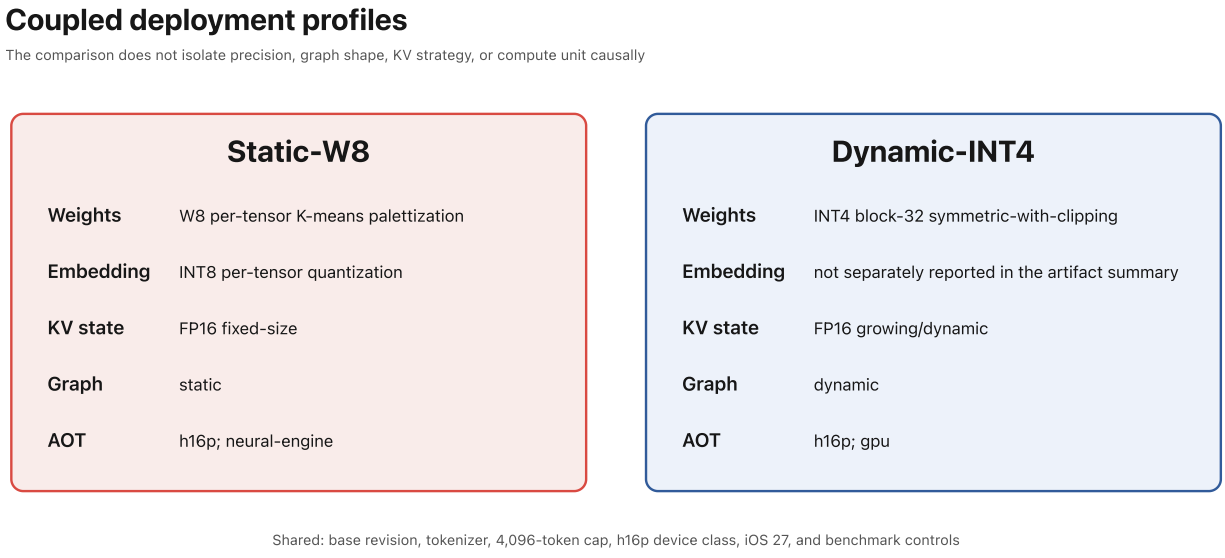

Figure 2: The compared profiles bundle multiple choices. Performance differences cannot be attributed causally to ANE versus GPU or W8 versus INT4 alone.

4 Methods

4.1 Study design and provenance

The paired CMRC2018 comparison is retrospective: its protocol, artifacts, and per-example predictions predate manuscript drafting. The fidelity-v2 evaluation and load-compatibility diagnostic use separate protocols. The current speed result comes from a prospectively governed eight-block acquisition admitted under Report Protocol V3. The paper pipeline verifies the inputs in `analysis/source-lock.json`, reconstructs the CMRC sample, repeats deterministic scoring, validates the public speed bundle, and generates the quantitative tables and figures. The pipeline itself performs no model inference or device measurement.

Repository commits, Hub revisions, and SHA-256 digests identify the inputs. The three W8 payloads implement the same frozen W8 mechanism but are not byte-identical. The historical artifact supports the CMRC2018 and historical device claims, the July public rebuild supports its six-case suite and later load failures, and the current public artifact supports the speed-v2 measurements.

4.2 W8 conversion-fidelity selection

W8 was selected with frozen uncompressed-reference logits and NLL rather than readable-text smoke tests. Uniform W4 and mixed W4/W8 candidates were rejected before the W8 mechanism was fixed; the historical selection record reports representative candidate mean cosines from 0.856886 to 0.981232.

A separate formal protocol evaluated the fixed mechanism with six tuning and four holdout inputs, identical serialized inputs for reference and candidate models, shared reference completions for teacher forcing, and case-level macro-averages computed in IEEE-754 binary64. Both models used the prospectively fixed numerical-parity setting described in Fidelity V2 Amendment 1. The candidate applied the published W8 recipe to 196 transformer projections and the iOS INT8 path to the tied embedding; the reference disabled both compression steps. The evaluation exercised the fake-palettized PyTorch authoring model on CPU, not the compiled device artifact.

The formal evaluation completed all 10 reference runs, 10 candidate runs, and 10 comparisons. Its values supersede the earlier W8 fidelity numbers in `results/quality-summary.json`; that file remains part of the historical model-selection record. The tuning and holdout inputs remained separate, and neither set is a downstream capability benchmark.

4.3 Physical-device validation

`A-W8-HISTORICAL` was tested on an iPhone 15 Pro (`iPhone16,1`, A17 Pro, `h16p`) running iOS 27. Each six-case suite covered structured output, multi-turn context, an instruction-injection boundary, a 3,790-token long-context retrieval, reasoning output, and unload/reload behavior. Four complete suites ran, and the final suite recorded zero hard failures.

An Instruments trace was summarized by counts of MPSGraph program and Apple Neural Engine Prediction intervals. The sanitized summary, rather than the original trace bundle, is public. The recorded intervals support ANE participation in the traced inference program.

`A-W8-JULY-PUBLIC` was rebuilt separately from the historical A/B binary to remove private path data. It passed a six-case suite on the same device class in its recorded environment; the two artifacts retain separate identities.

We later ran a descriptive load-only protocol on an iPhone 15 Pro with iOS 27 build `24A5424a`. It comprised two attempts with `A-W8-JULY-PUBLIC`—the second after a full-reboot request and observed reconnection—and one load-and-unload attempt with `A-W8-CURRENT`. The reboot command timed out while waiting for the device, so completion of the reboot was not directly verified from a boot-session identifier or uptime. That diagnostic collected no generation or Instruments trace, and cache state was not controlled; it therefore defines neither a cold-start benchmark nor a performance comparison. The later speed-v2 run evaluated `A-W8-CURRENT` under a separate protocol.

4.4 Paired CMRC2018 quality comparison

CMRC2018 is a span-extraction Chinese machine-reading-comprehension dataset (Cui et al., 2019). The paired experiment used a deterministic 300-example subset of its validation split: one question per unique context and 100 examples from each predefined context-length range (276–417, 418–630, and 631–980 characters), interleaved in a fixed order. Equal allocation gives the three length bands equal weight; it does not reproduce the validation split's natural length distribution. The historical protocol records the cut points but no separate statistical derivation for them, so all inferences are limited to this frozen, length-balanced subset. Its SHA-256 is `9fb9d896c96fc2ef0fa9961a5b65d2ac5b8bd09744f717a0a0ecb9f6c9fe05ff`.

Both profiles used the same tokenizer, chat template with `enable_thinking=false`, prompt, order, 128-token response budget, temperature zero, and a fresh session per example. Raw generated text was retained. The paper pipeline rebuilt 3,219 canonical rows from official repository commit `c0eb1b6ba219847457e6af3180da722bbeb656af`, reconstructed the exact frozen sample, and reproduced the published scoring JSON byte-for-byte under Python 3.11.15 and NLTK 3.10.0. Because the original CMRC scorer targeted Python 2, this result is explicitly treated as a Python-3 reanalysis.

The primary metrics use the official CMRC mixed Chinese/English EM/F1 procedure, whose `find_lcs` implementation scores the longest contiguous common token span. A subsequent uncertainty analysis used a stratified paired bootstrap aligned with the subset's equal allocation across the short, medium, and long context bands. Each of 10,000 resamples draws 100 examples with replacement within each band; F1 uses seed 20260721, EM uses seed 20260722, and percentile endpoints use Hyndman–Fan type 7 interpolation. We also report exact two-sided paired sign tests after excluding ties (Efron, 1979). An interval containing zero is treated as inconclusive; we performed no equivalence or non-inferiority test.

The initial feasibility protocol also required Dynamic-INT4 to remain within 1.0 point of Static-W8 on both EM and F1. The observed Dynamic-INT4-minus-Static-W8 EM difference was -1.67 points, so this heuristic gate failed. The separately specified paired analysis yielded intervals that included zero.

4.5 Workload-specific latency and memory

The latency experiment compares the two complete public profiles rather than isolated hardware units. Both variants used the same tokenizer, chat template, prompts, non-thinking control, temperature, and fresh sessions. The frozen schedule alternated eight physical profile blocks. Each profile contributed four blocks and 20 completed measurements per workload, for 120 observations and 60 paired logical samples overall.

The three workload groups were a medium business prompt, near-4K prefill with a short response, and a 256-token decode cap. Median input/output counts were 150/56 for Static-W8 and 150/22 for Dynamic-INT4 in the business workload, 3,778/6 for both profiles in the near-4K workload, and 108/256 for both profiles in the sustained-decode workload.

Reported latency values are medians of the 20 completed measurements in each profile/workload cell, not P95 estimates. Peak RSS is the maximum after-unload process peak across the four physical blocks for each profile; it is neither total system RAM nor a universal minimum-memory requirement.

Token TTFT ends at the first output-token event, whereas visible TTFT ends at the first nonempty visible-text snapshot. Visible decode throughput excludes the first visible snapshot and the preceding latency; end-to-end visible throughput divides final visible tokens by total generation time.

5 Results

5.1 W8 fidelity and authoring validation

Table 3 reports the corrected tuning and holdout conversion metrics. The holdout mean logit cosine was 0.996897, the minimum was 0.953391, top-1 agreement was 96.97%, and the mean candidate-minus-reference NLL delta was 0.008880.

Evaluation	Mean cosine	Min. cosine	Top-1 agree.	Mean ΔNLL
Tuning	0.997300	0.963419	98.96%	0.004190
Frozen holdout	0.996897	0.953391	96.97%	0.008880

Table 3: W8 authoring-model conversion fidelity relative to the frozen uncompressed reference. Values are case-level macro-averages from the corrected formal fidelity-v2 run, not compiled-device logits or a capability benchmark.

On the later Apple-main revision, patch application, focused tests, related test collection, lint, preset resolution, a full export, and AOT compilation all completed. Appendix A reports the repeated-export observation and exact identities.

5.2 Device execution and trace evidence

The historical artifact completed four full six-case suites. Its final suite loaded in 11.312 seconds after specialization-cache warm-up, peaked at 2,963.7 MiB process RSS, and returned to 198.7 MiB after unload. The long-context case consumed 3,790 input tokens, retrieved its tail marker, and produced a first token in 5.627 seconds.

The trace contained 547 MPSGraph program intervals and 547 Apple Neural Engine Prediction intervals. The exact public rebuild separately passed all six cases in its recorded environment; it peaked at 2,766.4 MiB process RSS and produced the near-4K first token in 5.709 seconds.

In the compatibility diagnostic on iOS 27 build `24A5424a`, `A-W8-JULY-PUBLIC` failed at `ANECCompileOffline` in two load-only attempts. `A-W8-CURRENT` completed its single load in 41.932 seconds, reached 2,737.0 MiB peak process RSS, unloaded, and exited normally. Those observations are load-only; the later speed-v2 measurements are reported separately.

Evidence object	Measure A	Measure B	Interpretation / identity
Historical suites	4 complete	6 per suite	Pass; zero recorded hard failures
Historical final suite	2,963.7 MiB peak RSS	198.7 MiB after unload	A-W8-HISTORICAL
Historical trace	547	547	ANE participation; non-exclusive
Public rebuild suite	6 / 6 cases	2,766.4 MiB peak RSS	A-W8-PUBLIC
Public near-4K case	3,790 input / 10 output	5.709 s TTFT	Tail marker retrieved

Table 4: Physical-device evidence on iPhone 15 Pro / iOS 27. Historical and public W8 payloads use the same mechanism but are not byte-identical.

5.3 Paired quality

All 300 examples completed for both profiles without hitting the 128-token budget. W8 produced 178 exact matches (59.33 EM) and 81.70 F1; INT4 produced 173 exact matches (57.67 EM) and 81.42 F1. The generated responses were byte-identical on 187 of 300 examples.

The observed Static-W8-minus-Dynamic-INT4 difference was +1.67 EM points with a 95% stratified bootstrap interval of [-3.00, +6.33] and +0.28 F1 points with an interval of [-2.47, +2.96]. After excluding ties, the exact two-sided paired sign-test p-values were 0.576 for EM and 0.614 for F1. Both intervals include zero.

Profile	n	EM	F1	Exact matches
Static-W8	300	59.33	81.70	178
Dynamic-INT4	300	57.67	81.42	173
Static-W8 − Dynamic-INT4	300	+1.67	+0.28	–

W8-minus-INT4 95% bootstrap intervals: EM [-3.00, +6.33], F1 [-2.47, +2.96]. Sign-test p: EM 0.576, F1 0.614.

Table 5: Paired CMRC2018 quality on the frozen 300-example subset. Both intervals include zero; no equivalence test was performed.

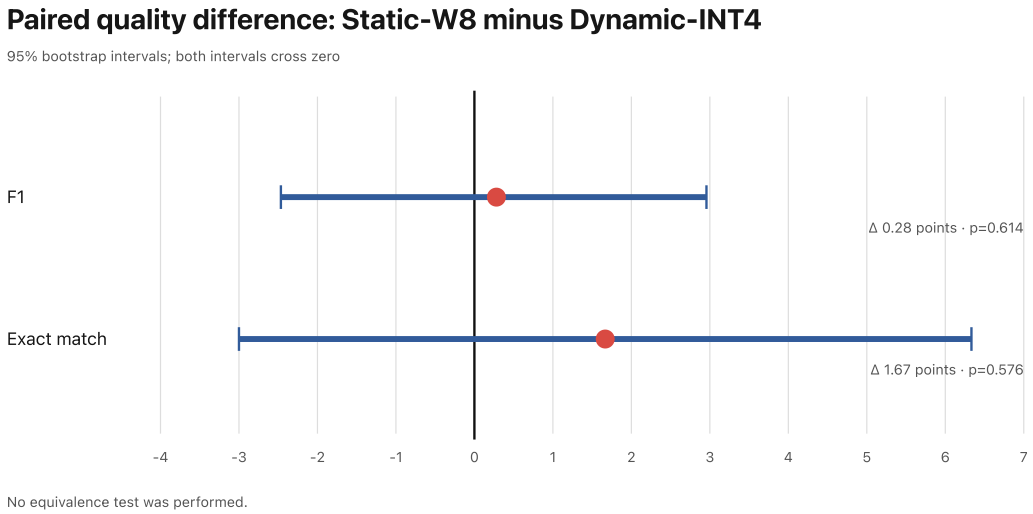

Figure 3: Paired Static-W8-minus-Dynamic-INT4 quality differences with 95% bootstrap intervals.

5.4 Storage and process memory

The current public W8 compiled resource directory occupied 1,711.4 MiB, compared with 924.6 MiB for INT4; INT4 was 786.8 MiB smaller (46.0%). Peak process RSS was 3,409.1 MiB for W8 and 1,978.0 MiB for INT4; INT4 used 1,431.2 MiB less (42.0%) in the disclosed benchmark.

Metric (MiB)	Static-W8	Dynamic-INT4	Dynamic-INT4 reduction
Compiled-model storage	1,711.4	924.6	786.8 (46.0%)
Peak process RSS	3,409.1	1,978.0	1,431.2 (42.0%)

Table 6: Compiled-model storage is the median of four identical estimatedDiskBytes observations converted to MiB; peak process RSS is the maximum after-unload process peak across four physical blocks.

5.5 Workload-dependent latency

For the business workload, Static-W8 reached the first token in 0.242 seconds versus 1.643 seconds for Dynamic-INT4. Dynamic-INT4 decoded faster (45.841 versus 27.542 visible tokens/s). Its median total time was also slightly lower (2.099 versus 2.234 seconds), but the median response lengths differed substantially (22 versus 56 tokens), so total duration is not a fixed-output comparison.

For the near-4K workload, both profiles produced a median of six tokens from 3,778 input tokens. Static-W8 reduced TTFT from 13.393 to 3.720 seconds and total time from 13.522 to 3.952 seconds. Because only five visible tokens follow the first visible token, TTFT and total time are more informative here than decode rate.

For the 108/256 sustained-decode workload, Dynamic-INT4 reduced total time from 9.675 to 6.454 seconds and increased visible decode from 26.931 to 42.210 tokens/s. Static-W8 retained the lower median TTFT, 0.205 versus 0.390 seconds.

Input / output	Profile	TTFT_t (s)	TTFT_v (s)	Total (s)	Decode	E2E
150 / 56	Static-W8	0.242	0.242	2.234	27.542	25.066
150 / 22	Dynamic-INT4	1.643	1.643	2.099	45.841	10.479
3,778 / 6	Static-W8	3.720	3.720	3.952	21.394	1.518
3,778 / 6	Dynamic-INT4	13.393	13.393	13.522	38.556	0.444
108 / 256	Static-W8	0.205	0.205	9.675	26.931	26.459
108 / 256	Dynamic-INT4	0.390	0.390	6.454	42.210	39.760

Table 7: Workload medians from 20 completed measurements per profile and workload across four physical blocks. Input and output are median token counts. TTFT_t is token-event TTFT, TTFT_v is visible-token TTFT, and Decode and E2E are visible tokens per second; no P95 is reported. Static-W8 is A-W8-CURRENT at Hub revision `75bbe06906cb5d953e602e3e4fb6364187c81822`.

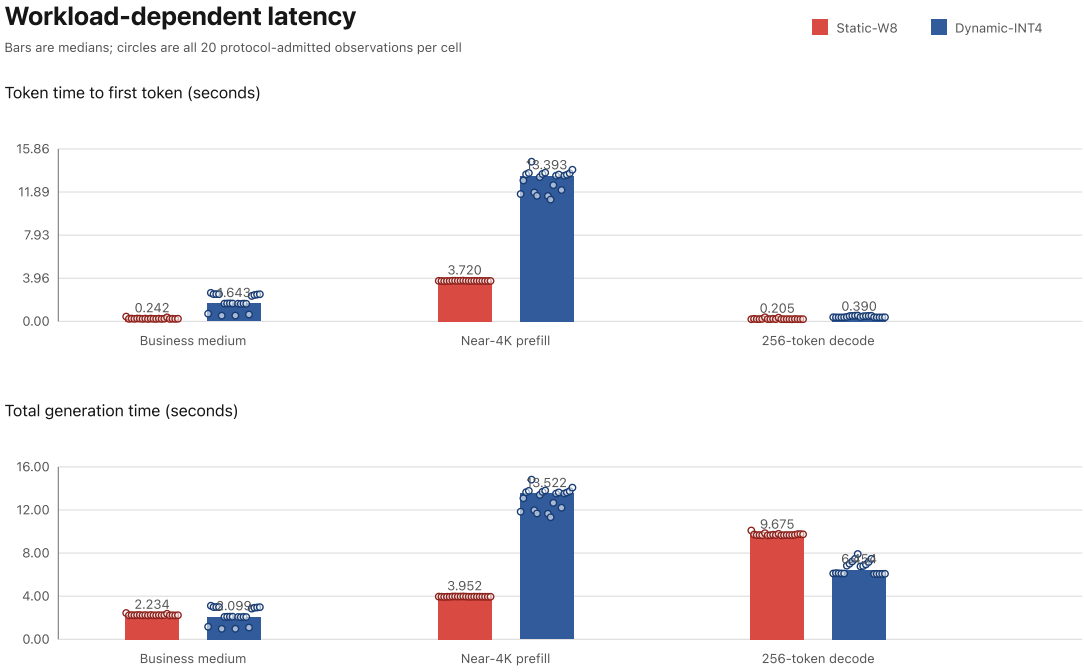

Figure 4: TTFT and total generation time for the three workload groups. Bars show medians; circles show all 20 completed measurements in each profile/workload cell.

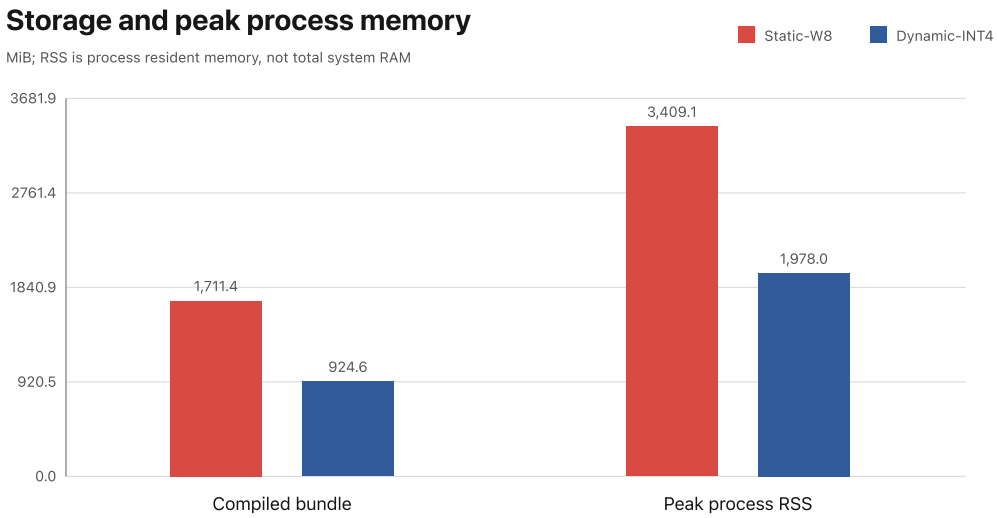

Figure 5: Compiled bundle size and peak process resident memory. RSS is not total system RAM.

Dynamic-INT4 used less storage and peak RSS and decoded faster in the sustained-generation workload. Static-W8 produced the first token sooner in all three workloads and completed the near-4K workload in less time.

6 Discussion

6.1 Workload-dependent trade-offs

The measurements show different advantages at different prompt and response lengths. Dynamic-INT4 has higher visible-decode medians in all three workloads and the advantage is most directly comparable in the 256-token workload. Static-W8 reaches the first token sooner in all three workloads and completes the long-prefill, short-output workload sooner. The business responses have different median lengths, so their total times do not establish a fixed-output advantage. These observations cover three workload shapes and do not define a general crossover point.

The profiles also change several factors together: weight representation, embedding treatment, graph shape, KV allocation, engine routing, artifact layout, and memory behavior. The results therefore characterize the two complete profiles rather than the isolated effect of precision or compute unit.

6.2 Scope of the quality comparison

The paired CMRC2018 experiment measures extractive Chinese reading comprehension on a length-balanced subset. Its confidence intervals include zero, and the experiment was not designed to establish equivalence. Applying either profile to summarization, classification, structured output, instruction-boundary handling, or bookmark analysis requires evaluation on that task.

6.3 Artifact and runtime compatibility

Two non-identical artifacts produced by different compiler builds had different load outcomes in one runtime. This comparison cannot identify whether artifact bytes, compiler producer, OS/runtime state, or memory caused the failures.

7 Reproducibility

The public W8 repository is fixed at commit `46668db811559ce21b0006f07706a6d0bde08656`, and the paired-comparison repository at `34a4c08a9282bd076b8b5fe154c5507e6a8b3774` (Shi, 2026e,a). The historical quality and device records bind source SHA-256 `66325b4cc0657e1f89a7a0a92f37899d01ddc37d168295dbbad0d71bef7f75e3` and compiled-main SHA-256 `0c2dfcfeaae195386f1e61c05e0cf2b4a1ce6ecda1c321803afc0019b1d886d7`. The July public six-case record uses Hub revision `466ebe2e5cec125fa113ea71503add41bba581a8`; model-card revision `be6b5aad19e18bb71be3199eadceb27db7e69724` removes a local device nickname without changing that payload or validation file. The current speed run uses W8 branch `xcode27-beta6-aot` at revision `75bbe06906cb5d953e602e3e4fb6364187c81822` and INT4 revision `c32b6342c98e5e23363f692e614bccca37f24234`. The historical quality comparison recorded the INT4 documentation-only predecessor `cb25b3226ee679ee92d0e5af7467d579a6bff66a`, whose compiled payload hashes are unchanged.(Shi, 2026d,c,b)

The W8 recipe SHA-256 is `dab03ae1dd6c6290a7964e05ebcda7fe027c2bb240174fcf034e64a376be9d72`. The July and current W8 compiled `main-h16p.mlirb` hashes are `a7eefeef16708a324f9919890355eb92180ec85eef419ebd5822e8c8afd42f5f` and `09f609775baa56b11ff3c91bfcb07b145930297289634fdc5514b2a5ab4dc7ca`, respectively; the accepted Run J download manifest is `91c68e82e280a36d39aaeef9b8726ab1d59e52760b6f970db67c334a674d47b2`. The INT4 compiled main and `resources.bin` hashes are `ad953b6bc902accc1f1200a8870012c5dec6b488f0f4ec0f10a9916b16cb56ef` and `37c7f2fcf85625abb32d83ca24e7a9facd33a37f79dbc3afec64d41eb900f8bc`.

The repository separates evidence reconstruction from document compilation. Running `./analysis/run.sh` fetches the commits and remote objects listed in `analysis/source-lock.json`, verifies the corrected fidelity-v2 and compatibility summaries, reconstructs the CMRC reference sample, recomputes published CMRC quality, validates the finalized speed-v2 public bundle and all 120 observations, and regenerates Tables 1–8 and Figures 3–5. It pins Python 3.11.15 and NLTK 3.10.0. Running `./manuscript/build.sh` then checks the committed generated outputs and compiles the paper with the documented Chrome and Tectonic versions. The reconstructed CMRC quality JSON is byte-identical to the published Python-3 result.

The dataset text is not redistributed by the paper pipeline. Instead, the official CMRC source commit and SHA-256, canonical-row SHA-256, and frozen-sample SHA-256 are checked before scoring.

8 Limitations and Threats to Validity

Hardware and toolchain. All device evidence is from one iPhone 15 Pro / A17 Pro / `h16p` class. The historical suites, paired CMRC2018 comparison, and current speed run each use their recorded iOS 27 and Xcode 27 prerelease environments. No other device class was evaluated.

Coupled profiles. The system profiles differ in precision, compression granularity, embedding treatment, graph shape, KV allocation, preferred compute, and artifact layout. The evaluation therefore supports profile-level comparison, not causal attribution to ANE, GPU, W8, or INT4.

Execution trace. Only the sanitized historical trace summary is distributed. It records ANE participation for the traced historical artifact but cannot be used to reconstruct event-level scheduling or exclusive dispatch.

Quality scope. The fidelity-v2 result covers ten frozen authoring-model inputs and does not measure logits from the compiled iPhone artifact. The paired quality run contains a length-balanced subset of 300 CMRC2018 examples rather than the full validation split or its natural length distribution. Confidence intervals include zero, and no equivalence margin was specified. The initial 1.0-point EM heuristic failed. Complete WikiText-2 perplexity and downstream task evaluations are unavailable.

Timing scope. Each profile/workload cell reports 20 completed measurements across four physical blocks, but the protocol does not estimate tail latency or P95. The business responses differ in median output length, so their total times are not fixed-output comparisons. The near-4K response contains only six visible tokens, making its decode-rate estimate secondary to TTFT and total time.

Energy and lifecycle. Energy, battery use, thermal endurance, background execution, and Jetsam behavior were outside the measurement protocol.

Compatibility diagnostic. The supplementary compatibility check used one device/runtime, two attempts with `A-W8-JULY-PUBLIC`, and one with `A-W8-CURRENT`. It included no generation or trace, and cache state was uncontrolled; its design therefore precludes causal attribution. Two authoring exports made from the same pinned inputs and parameters but different output directories were not byte-identical. No semantic or numerical comparison was made between those exports.

9 Conclusion

We implemented a version-pinned static Apple Core AI path for Qwen3-1.7B, compiled it for `h16p`, and ran the historical artifact through `LanguageModelSession` on an iPhone 15 Pro with measured ANE participation. A formal evaluation under the amended fidelity-v2 protocol confirmed high conversion fidelity for the frozen W8 authoring mechanism while keeping compiled-device behavior outside that claim. In the same-device profile comparison, Dynamic-INT4 used less storage and process memory and decoded faster, whereas Static-W8 reached the first token sooner and completed the long-prompt workload faster.

10 Data and Code Availability

The implementation, experiment protocols, deterministic analysis, and manuscript sources are available in the public project repository (Shi, 2026e). The July and current W8 revisions and the INT4 artifact are available from the cited Hugging Face repositories (Shi, 2026d,c,b); paired predictions and benchmark materials are available in the comparison repository (Shi, 2026a). Exact revisions and hashes are listed above. CMRC2018 text is not redistributed; the analysis reconstructs the sample from the official source revision. The load-only compatibility observation remains documented separately from the later speed run.

11 Author Contributions

Yi Shi led the software implementation, experiments, evidence analysis, and manuscript preparation. Mengni Wu contributed to study design, interpretation of the results, and manuscript review. Both authors approved the final manuscript.

12 Competing Interests

The authors declare no competing interests.

References

Apple Inc. Core AI models. GitHub repository, commit 7062017c8e86c6cf4f49b721ddc3494efcdb7c7d, 2026a. URL https://github.com/apple/coreai-models. Accessed 2026-08-28.

Apple Inc. Palettization overview: Guide to Core ML Tools. Core ML Tools documentation, 2026b. URL https://apple.github.io/coremltools/docs-guides/source/opt-palettization-overview.html. Accessed 2026-08-28.

Apple Inc. Add Qwen3-1.7B support for iOS. GitHub issue 116, 2026c. URL https://github.com/apple/coreai-models/issues/116. State audited 2026-08-28.

Apple Inc. Add Qwen3-1.7B iOS and macOS export support. GitHub pull request 196, 2026d. URL https://github.com/apple/coreai-models/pull/196. Closed without merge; state audited 2026-08-28.

Apple Machine Learning Research. Deploying transformers on the Apple Neural Engine. Apple Machine Learning Research, 2022. URL https://machinelearning.apple.com/research/neural-engine-transformers.

Yiming Cui, Ting Liu, Wanxiang Che, Li Xiao, Zhipeng Chen, Wentao Ma, Shijin Wang, and Guoping Hu. A span-extraction dataset for chinese machine reading comprehension. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 5883–5889, 2019. doi: 10.18653/v1/D19-1600. URL https://aclanthology.org/D19-1600/.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

kevinqz. Qwen3-1.7B Core AI stateful INT8 artifact. Hugging Face model card, revision 340f4c4c8c422c56e0cc9b5ecb7eabdf37781f54, 2026. URL https://huggingface.co/kevinqz/Qwen3-1.7B-CoreAI. Community artifact; not published or endorsed by Apple.

mlboydaisuke. Qwen3-1.7B Apple Core AI export for iPhone GPU. Hugging Face model card, revision af6335a8a108e383be8aa44925a8fe03a1cd2cd9, 2026. URL https://huggingface.co/mlboydaisuke/qwen3-1.7b-CoreAI-official. Community artifact; not published or endorsed by Apple.

Qwen Team. Qwen3-1.7B. Hugging Face model repository, revision 70d244cc86ccca08cf5af4e1e306ecf908b1ad5e, 2025. URL https://huggingface.co/Qwen/Qwen3-1.7B.

Yi Shi. Qwen3-1.7B Core AI reproduction and paired profile evaluation. GitHub repository, commit 34a4c08a9282bd076b8b5fe154c5507e6a8b3774, 2026a. URL https://github.com/massif-01/qwen3-1.7b-coreai-reproduction.

Yi Shi. Qwen3-1.7B Core AI GPU INT4 4K h16p artifact. Hugging Face model repository, revision c32b6342c98e5e23363f692e614bccca37f24234, 2026b. URL https://huggingface.co/massif/Qwen3-1.7B-CoreAI-GPU-INT4-4K-h16p.

Yi Shi. Qwen3-1.7B Core AI ANE W8 4K h16p: Current run j artifact. Hugging Face model repository, revision 75bbe06906cb5d953e602e3e4fb6364187c81822, 2026c. URL https://huggingface.co/massif/Qwen3-1.7B-CoreAI-ANE-W8-4K-h16p/tree/75bbe06906cb5d953e602e3e4fb6364187c81822. Compiled main SHA-256 09f609775baa56b11ff3c91bfcb07b145930297289634fdc5514b2a5ab4dc7ca.

Yi Shi. Qwen3-1.7B Core AI ANE W8 4K h16p: July public artifact. Hugging Face model repository, revision 466ebe2e5cec125fa113ea71503add41bba581a8, 2026d. URL https://huggingface.co/massif/Qwen3-1.7B-CoreAI-ANE-W8-4K-h16p/tree/466ebe2e5cec125fa113ea71503add41bba581a8. Compiled main SHA-256 a7eefeef16708a324f9919890355eb92180ec85eef419ebd5822e8c8afd42f5f.

Yi Shi. Qwen3-1.7B Core AI iOS: Reproducible W8 static-path implementation. GitHub repository, commit 46668db811559ce21b0006f07706a6d0bde08656, 2026e. URL https://github.com/massif-01/qwen3-1.7b-coreai-ios.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

A Supplementary W8 compatibility observation

Table 8 records a descriptive, load-only diagnostic conducted after the main experiments. It is not part of the historical A/B dataset, and its load duration is not treated as a cold-start or comparative-performance measurement.

Artifact	Producer; compiled-main SHA	Loads	Observation
Earlier public AOT	coreai-build-3600.75.3; a7eefeef1670...	2	Both stopped during ANECCompileOffline; the second followed a full-reboot request and observed reconnection
Current W8	coreai-build-3600.83.1; 09f609775baa...	1	Load 41.932 s; peak process RSS 2,737.0 MiB; unload completed; exit 0

Table 8: Supplementary load-only compatibility observations on one iPhone 15 Pro running iOS 27 build 24A5424a. The current-W8 result is not a cold-start benchmark, generation test, or Instruments trace; cache state was uncontrolled, and artifact bytes and compiler producer both changed.

Output	Identity type	SHA-256
`A-W8-CURRENT`	Authoring `main.mlirb`	`13ba3f73fcb7e090cd6ba1ca14b6b8903516ab608d451e94b9cdd750cfceda2c`
`A-W8-CURRENT`	Compiled main	`09f609775baa56b11ff3c91bfcb07b1459302972896 34fdc5514b2a5ab4dc7ca`
`A-W8-CURRENT`	`.aimodelc` file-list fingerprint	`182336f4654bb735bcad35e45f7832756c34469931ad96d872532dca727ebd8d`
Repeated authoring export	Authoring `main.mlirb`	`12349a9ad32bf2a1d2f9a6f201ffec3125c28c027786fa9925dc34669d8bc946`

Table 9: Exact identities for the current-toolchain W8 build and repeated authoring export.

The repeated export used the same pinned inputs and parameters but a different output directory. Its different digest establishes non-identical raw output in this two-run check, not a semantic or numerical difference.